

M223, Efe Bora Saglam, Amadeo Toma

Kaggle Tutorial Intro to ML 5P

1. How Models Work

Introduction

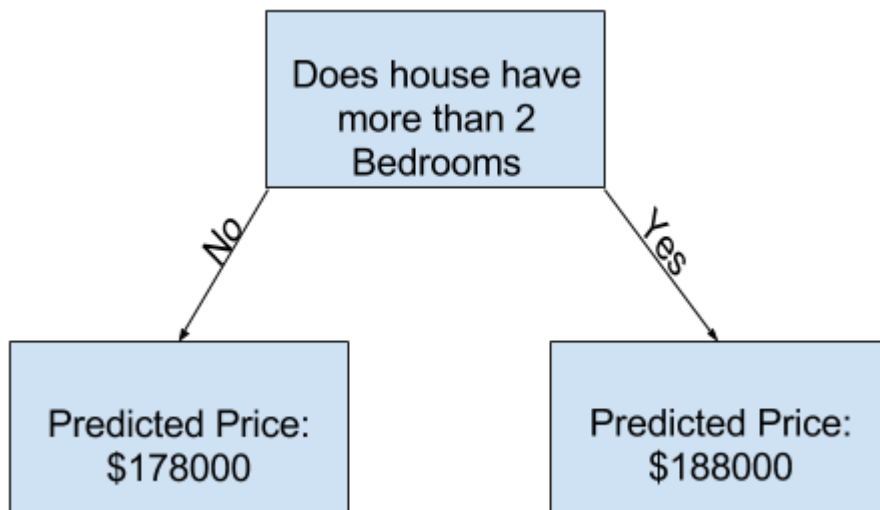
► Introduction

In diesem Kurs geht es zunächst darum zu verstehen, wie Machine-Learning-Modelle funktionieren und wie sie eingesetzt werden. Falls du schon Erfahrung mit Statistik oder Machine Learning hast, könnte dir der Anfang etwas einfach erscheinen – aber keine Sorge, wir arbeiten uns schnell zu komplexeren Modellen vor. Das Ganze wird anhand eines Szenarios erklärt: Dein Cousin hat durch Spekulation auf Immobilien Millionen verdient. Jetzt will er mit dir ein Geschäft starten – er bringt das Geld mit, du lieferst die Modelle, um den Wert von Häusern vorherzusagen.

Auf deine Frage, wie er bisher Immobilienwerte eingeschätzt hat, antwortet er mit „Intuition“. Bei genauerem Nachfragen stellt sich heraus, dass er aus Erfahrung Muster erkannt hat und diese nutzt, um neue Immobilien zu bewerten – genau das macht auch Machine Learning.

Wir starten mit einem einfachen Modell: dem Entscheidungsbaum (Decision Tree). Es gibt zwar genauere Modelle, aber Entscheidungsbäume sind leicht zu verstehen und bilden die Grundlage für viele der besten Verfahren im Machine Learning.

Zur Einführung fangen wir mit der einfachsten Form eines Entscheidungsbaums an.



Der Entscheidungsbaum teilt die Häuser in nur zwei Kategorien ein. Der vorhergesagte Preis für ein Haus ist dabei der durchschnittliche Preis aus der Vergangenheit für Häuser in derselben Kategorie.

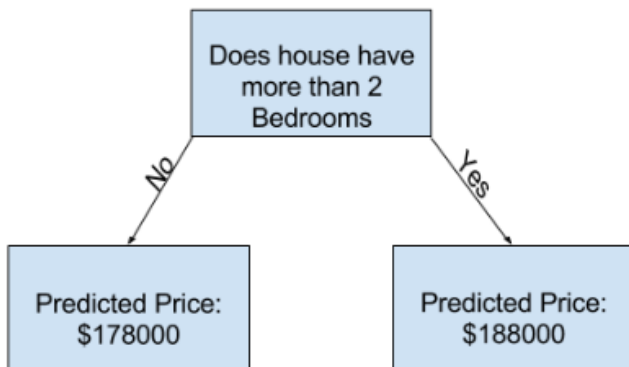
Um diese Einteilung vorzunehmen und die Vorhersagewerte zu bestimmen, wird vorhandene Daten verwendet. Dieser Vorgang, bei dem Muster aus den Daten erkannt werden, nennt sich „Training“ oder „Anpassen“ des Modells. Die dafür verwendeten Daten nennt man Trainingsdaten.

Wie genau das Modell trainiert wird (z. B. wie die Daten aufgeteilt werden), ist etwas komplexer und wird später erklärt. Sobald das Modell trainiert ist, kann es verwendet werden, um Vorhersagen für neue Häuser zu treffen.

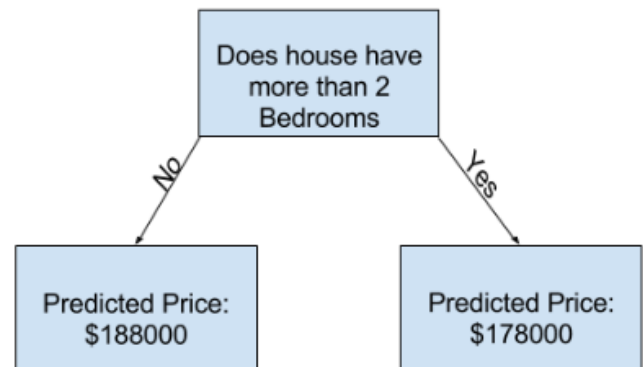
Improving the Decision Tree

► Improvng the Decision Tree

1st Decision Tree

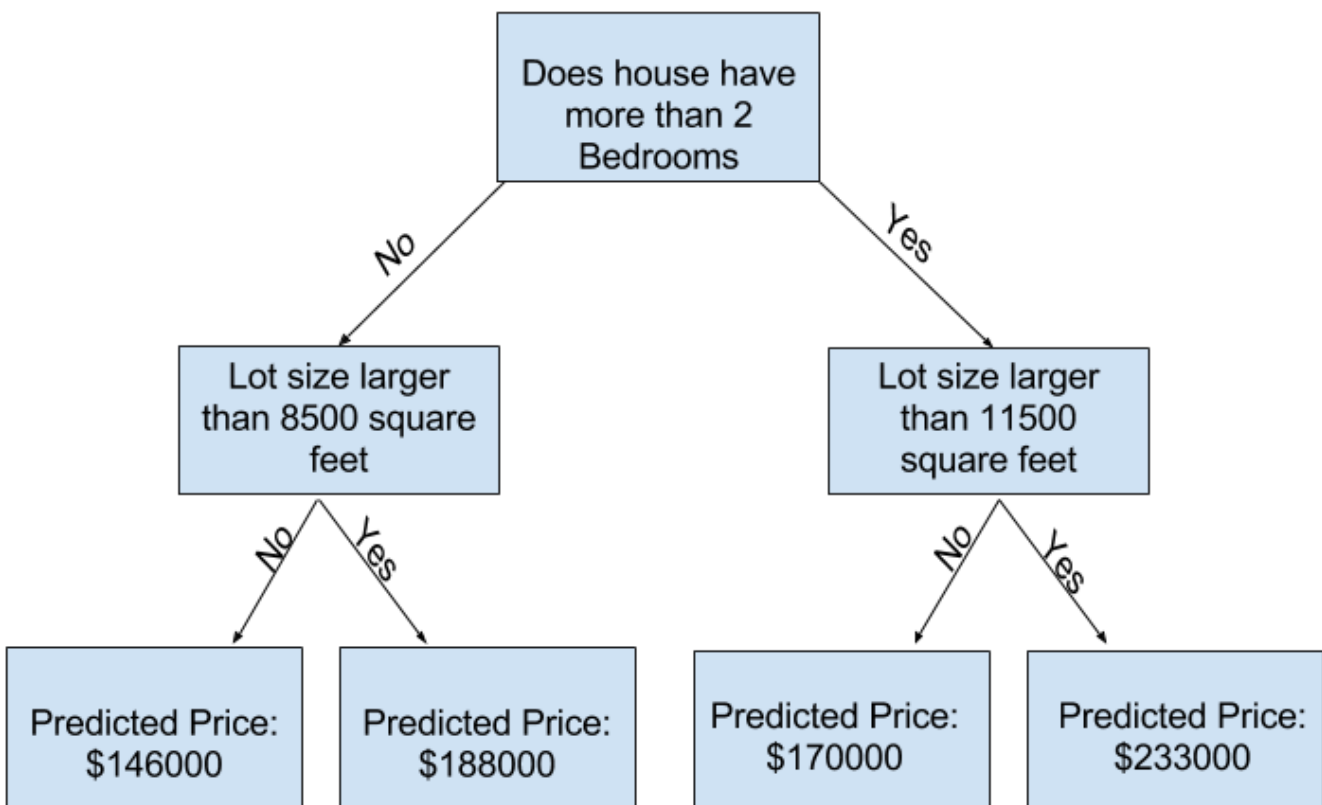


2nd Decision Tree



Der Entscheidungsbaum auf der linken Seite (Entscheidungsbaum 1) ist wahrscheinlich sinnvoller, weil er die Realität abbildet, dass Häuser mit mehr Schlafzimmern tendenziell teurer verkauft werden als solche mit weniger Schlafzimmern. Der größte Nachteil dieses Modells ist, dass es viele andere Faktoren, die den Hauspreis beeinflussen – wie Anzahl der Badezimmer, Grundstücksgröße, Lage usw. – nicht berücksichtigt.

Man kann mehr dieser Faktoren erfassen, indem man einen Baum mit mehr „Verzweigungen“ erstellt. Solche Bäume nennt man „tiefere“ Bäume. Ein Entscheidungsbaum, der zum Beispiel auch die Grundstücksgröße berücksichtigt, könnte so aussehen:



Du sagst den Preis eines Hauses voraus, indem du im Entscheidungsbaum den Pfad entlanggehst, der den Merkmalen des Hauses entspricht. Der vorhergesagte Preis steht am unteren Ende des Baums. Dieser Punkt, an dem die Vorhersage getroffen wird, nennt sich Blatt (engl. leaf).

Die Aufteilungen im Baum und die Werte an den Blättern werden durch die Daten bestimmt – es ist also Zeit, dir die Daten anzusehen, mit denen du arbeiten wirst.

2. Basic Data Exploratoion

► Using Pandas to Get Familiar With Your Data 🐼

```
import pandas as pd
```

Der wichtigste Teil der Pandas-Bibliothek ist das DataFrame. Ein DataFrame enthält die Art von Daten, die man sich als Tabelle vorstellen kann. Das ist ähnlich wie ein Arbeitsblatt in Excel oder eine Tabelle in einer SQL-Datenbank.

Pandas bietet leistungsstarke Methoden für fast alles, was du mit dieser Art von Daten tun möchtest.

Als Beispiel schauen wir uns Daten zu Immobilienpreisen in Melbourne, Australien, an. In den praktischen Übungen wirst du denselben Prozess auf einen neuen Datensatz anwenden, der Immobilienpreise in Iowa enthält.

Die Beispiel-Daten (Melbourne) befinden sich unter dem Dateipfad ../input/melbourne-housing-snapshot/melb_data.csv.

Wir laden und erkunden die Daten mit den folgenden Befehlen:

```
# save filepath to variable for easier access
melbourne_file_path = '../input/melbourne-housing-snapshot/melb_data.csv'
# read the data and store data in DataFrame titled melbourne_data
melbourne_data = pd.read_csv(melbourne_file_path)
# print a summary of the data in Melbourne data
melbourne_data.describe()
```

	Rooms	Price	Distance	Postcode	Bedroom2	Bathroom	Car	Landsize	Bu
count	13580.000000	1.358000e+04	13580.000000	13580.000000	13580.000000	13580.000000	13518.000000	13580.000000	71
mean	2.937997	1.075684e+06	10.137776	3105.301915	2.914728	1.534242	1.610075	558.416127	15
std	0.955748	6.393107e+05	5.868725	90.676964	0.965921	0.691712	0.962634	3990.669241	54
min	1.000000	8.500000e+04	0.000000	3000.000000	0.000000	0.000000	0.000000	0.000000	0.0
25%	2.000000	6.500000e+05	6.100000	3044.000000	2.000000	1.000000	1.000000	177.000000	93
50%	3.000000	9.030000e+05	9.200000	3084.000000	3.000000	1.000000	2.000000	440.000000	12
75%	3.000000	1.330000e+06	13.000000	3148.000000	3.000000	2.000000	2.000000	651.000000	17
max	10.000000	9.000000e+06	48.100000	3977.000000	20.000000	8.000000	10.000000	433014.000000	44

► Interpreting Data Description 🔍

Fehlende Werte entstehen aus verschiedenen Gründen. Zum Beispiel würde die Größe des zweiten Schlafzimmers bei einer Erhebung eines Ein-Zimmer-Hauses nicht erfasst werden. Wir werden später noch auf das Thema fehlende Daten zurückkommen.

Der zweite Wert ist der Durchschnitt (mean), also der Mittelwert. Darunter steht std, die Standardabweichung, die misst, wie stark die Werte numerisch verteilt sind.

Um die Werte min, 25%, 50%, 75% und max zu interpretieren, stelle dir vor, du sortierst jede Spalte von niedrigstem bis höchstem Wert. Der erste (kleinste) Wert ist das min. Wenn du ein Viertel der Liste durchgehst, findest du eine Zahl, die größer als 25% der Werte und kleiner als 75% der Werte ist. Das ist der 25%-Wert (auch als „25. Perzentil“ bezeichnet). Das 50. und 75. Perzentil werden entsprechend definiert, und das max ist die größte Zahl.

Basic Data Exploration exercises:

► Exercise

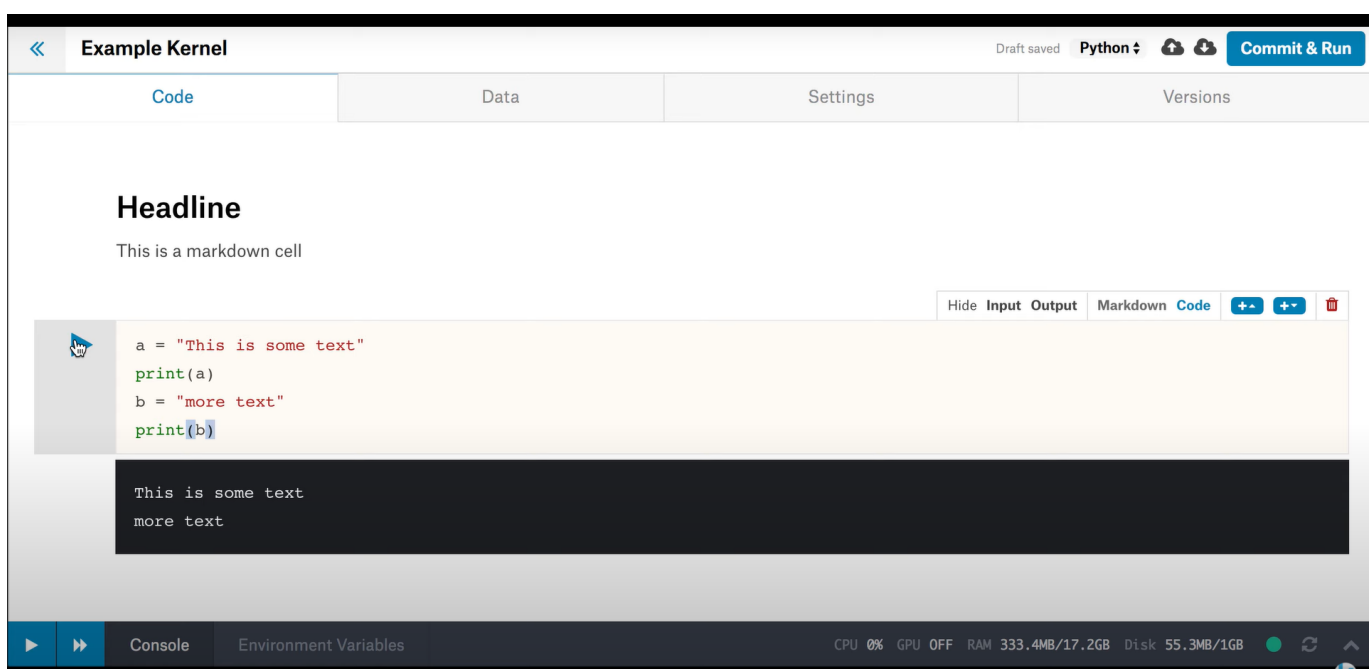
Diese Übung wird Ihre Fähigkeit testen, eine Datendatei zu lesen und Statistiken über die Daten zu verstehen.

In späteren Übungen werden Sie Techniken anwenden, um die Daten zu filtern, ein Machine-Learning-Modell zu erstellen und Ihr Modell schrittweise zu verbessern.

Die Kursbeispiele verwenden Daten aus Melbourne. Um sicherzustellen, dass Sie diese Techniken eigenständig anwenden können, werden Sie sie auf ein neues Datenset (mit Immobilienpreisen aus Iowa) anwenden müssen.

Die Übungen verwenden eine sogenannte "Notebook"-Programmierungsumgebung. Falls Sie mit Notebooks nicht vertraut sind, haben wir ein 90-sekündiges Einführungsvideo vorbereitet.

Führen Sie die folgende Zelle aus, um die Code-Überprüfung einzurichten, die Ihre Arbeit während des Fortschreitens überprüft.



The screenshot shows a Jupyter Notebook interface. At the top, there's a header bar with a back arrow, the text "Example Kernel", and a "Draft saved" indicator. To the right, there's a "Python" language selector, a "Commit & Run" button, and icons for saving and sharing. Below the header, there are four tabs: "Code" (active), "Data", "Settings", and "Versions". The main area contains a "Headline" section with the text "This is a markdown cell". Below this is a code cell with the following Python code:

```
a = "This is some text"
print(a)
b = "more text"
print(b)
```

The output of the code cell is displayed in a dark box below the code:

```
This is some text
more text
```

At the bottom of the notebook, there's a status bar showing "CPU 0%", "GPU OFF", "RAM 333.4MB/17.2GB", and "Disk 55.3MB/1GB". There are also icons for running, stopping, and refreshing the kernel.

```
# Set up code checking
from learntools.core import binder
binder.bind(globals())
from learntools.machine_learning.ex2 import *
print("Setup Complete")
```

Setup Complete

Step 1_ Loading Data

In der Aufgabenstellung steht:

"Read the Iowa data file into a Pandas DataFrame called home_data."

Das bedeutet: Du sollst die CSV-Datei einlesen und als DataFrame speichern, nicht einfach den Dateipfad als Text (`str`) speichern.

In deinem Code hast du geschrieben:

```
import pandas as pd

# Path of the file to read
iowa_file_path = '../input/home-data-for-ml-course/train.csv'

# Read the file into a variable home_data
home_data = pd.read_csv(iowa_file_path)

# Check if it is loaded correctly
step_1.check()
```

Kurz gesagt:

- `home_data` soll ein DataFrame sein (mit `pd.read_csv` geladen),
- nicht einfach ein Text/String

```
import pandas as pd

# Path of the file to read
iowa_file_path = '../input/home-data-for-ml-course/train.csv'

# Fill in the line below to read the file into a variable home_data
home_data = pd.read_csv(iowa_file_path)

# Call line below with no argument to check that you've loaded the data correctly
step_1.check()
```

Correct

Step 2: Review The Data

Verwenden Sie den Befehl, den Sie gelernt haben, um die Zusammenfassungsstatistiken der Daten anzuzeigen. Füllen Sie anschließend die Variablen aus, um die folgenden Fragen zu beantworten.

```
# Statistiken anzeigen
print(home_data.describe())
```

	Id	MSSubClass	LotFrontage	LotArea	OverallQual	\
count	1460.000000	1460.000000	1201.000000	1460.000000	1460.000000	
mean	730.500000	56.897260	70.049958	10516.828082	6.099315	
std	421.610009	42.300571	24.284752	9981.264932	1.382997	
min	1.000000	20.000000	21.000000	1300.000000	1.000000	
25%	365.750000	20.000000	59.000000	7553.500000	5.000000	
50%	730.500000	50.000000	69.000000	9478.500000	6.000000	
75%	1095.250000	70.000000	80.000000	11601.500000	7.000000	
max	1460.000000	190.000000	313.000000	215245.000000	10.000000	

	OverallCond	YearBuilt	YearRemodAdd	MasVnrArea	BsmtFinSF1	...
count	1460.000000	1460.000000	1460.000000	1452.000000	1460.000000	...
mean	5.575342	1971.267808	1984.865753	103.685262	443.639726	...
std	1.112799	30.202904	20.645407	181.066207	456.098091	...
min	1.000000	1872.000000	1950.000000	0.000000	0.000000	...
25%	5.000000	1954.000000	1967.000000	0.000000	0.000000	...
50%	5.000000	1973.000000	1994.000000	0.000000	383.500000	...
75%	6.000000	2000.000000	2004.000000	166.000000	712.250000	...
max	9.000000	2010.000000	2010.000000	1600.000000	5644.000000	...

	WoodDeckSF	OpenPorchSF	EnclosedPorch	3SsnPorch	ScreenPorch	\
count	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	
mean	94.244521	46.660274	21.954110	3.409589	15.060959	
std	125.338794	66.256028	61.119149	29.317331	55.757415	
min	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	0.000000	0.000000	0.000000	0.000000	0.000000	
50%	0.000000	25.000000	0.000000	0.000000	0.000000	
75%	168.000000	68.000000	0.000000	0.000000	0.000000	
max	857.000000	547.000000	552.000000	508.000000	480.000000	

	PoolArea	MiscVal	MoSold	YrSold	SalePrice
count	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000
mean	2.758904	43.489041	6.321918	2007.815753	180921.195890
std	40.177307	496.123024	2.703626	1.328095	79442.502883
min	0.000000	0.000000	1.000000	2006.000000	34900.000000
25%	0.000000	0.000000	5.000000	2007.000000	129975.000000
50%	0.000000	0.000000	6.000000	2008.000000	163000.000000
75%	0.000000	0.000000	8.000000	2009.000000	214000.000000
max	738.000000	15500.000000	12.000000	2010.000000	755000.000000

```
import datetime
# Durchschnittliche Grundstücksfläche (lotArea)
avg_lot_size = round(home_data['LotArea'].mean())

# As of today, how old is the newest home (current year - the date in which it was
built)
current_year = datetime.datetime.now().year
newest_home_age = current_year - home_data['YearBuilt'].max()

# Checks your answers
step_2.check()
```

3. Selecting Data for Modeling

► Selecting Data for Modeling

Wir beginnen damit, ein paar Variablen nach unserem Bauchgefühl auszuwählen. In späteren Kursen lernen Sie statistische Techniken kennen, um Variablen automatisch nach ihrer Wichtigkeit zu priorisieren.

Um Variablen/Spalten auszuwählen, müssen wir zunächst eine Liste aller Spalten im Datensatz sehen. Das geschieht über die `columns`-Eigenschaft des DataFrames (siehe letzte Zeile im folgenden Code).

```
import pandas as pd

melbourne_file_path = '../input/melbourne-housing-snapshot/melb_data.csv'
melbourne_data = pd.read_csv(melbourne_file_path)
melbourne_data.columns
```

```
Index(['Suburb', 'Address', 'Rooms', 'Type', 'Price', 'Method', 'SellerG',
       'Date', 'Distance', 'Postcode', 'Bedroom2', 'Bathroom', 'Car',
       'Landsize', 'BuildingArea', 'YearBuilt', 'CouncilArea', 'Latitude',
       'Longitude', 'Regionname', 'Propertycount'],
      dtype='object')
```

```
# Die Melbourne-Daten enthalten einige fehlende Werte (bei manchen Häusern wurden
# einige Variablen nicht erfasst).
# Wir werden später in einem Tutorial lernen, wie man mit fehlenden Werten umgeht.
# Ihre Iowa-Daten haben jedoch keine fehlenden Werte in den verwendeten Spalten.
# Daher wählen wir jetzt die einfachste Lösung und entfernen Häuser mit fehlenden
# Werten aus unseren Daten.
# Machen Sie sich darüber im Moment keine großen Gedanken. Der entsprechende Code
# lautet:

# dropna entfernt fehlende Werte (denken Sie bei na an "not available")
melbourne_data = melbourne_data.dropna(axis=0)
```

Es gibt viele Möglichkeiten, eine Teilmenge Ihrer Daten auszuwählen. Der Pandas-Kurs behandelt diese Methoden ausführlicher, aber wir konzentrieren uns zunächst auf zwei Ansätze:

- **Punktnotation**, die wir verwenden, um das **Vorhersageziel** auszuwählen.
- **Auswahl mit einer Spaltenliste**, die wir verwenden, um die **Merkmale (Features)** auszuwählen.

Selecting The Prediction Target

Sie können eine Variable mit Punktnotation herausziehen. Diese einzelne Spalte wird in einer **Series** gespeichert, die im Wesentlichen wie ein **DataFrame** ist, aber nur eine einzelne Datenspalte enthält.

Wir verwenden die Punktnotation, um die Spalte auszuwählen, die wir vorhersagen möchten – das sogenannte **Vorhersageziel**. Üblicherweise wird das Vorhersageziel **y** genannt.

Der Code, den wir benötigen, um die Hauspreise aus den Melbourne-Daten zu speichern, lautet:

```
y = melbourne_data.Price
```

Chosing "Feathures"

Die Spalten, die in unser Modell eingegeben werden (und später zur Vorhersage verwendet werden), werden als **"Features"** bezeichnet. In unserem Fall wären das die Spalten, die verwendet werden, um den Hauspreis zu bestimmen. Manchmal verwenden Sie alle Spalten außer dem Ziel als Features. In anderen Fällen ist es besser, mit weniger Features zu arbeiten.

Für den Moment werden wir ein Modell mit nur wenigen Features erstellen. Später werden Sie lernen, wie man Modelle mit unterschiedlichen Features iteriert und vergleicht.

Wir wählen mehrere Features aus, indem wir eine Liste von Spaltennamen innerhalb von eckigen Klammern bereitstellen. Jedes Element in dieser Liste sollte ein String (mit Anführungszeichen) sein.

Hier ist ein Beispiel:

```
melbourne_features = ['Rooms', 'Bathroom', 'Landsize', 'Lattitude', 'Longtitude']
```

Bei Konvention, dieses Data heisst X.

```
X = melbourne_data[melbourne_features]
```

Lassen Sie uns schnell die Daten durchsehen, die wir verwenden werden, um Hauspreise vorherzusagen, indem wir die **describe**-Methode und die **head**-Methode verwenden, die die obersten paar Zeilen anzeigt.

```
X.describe()
```

	Rooms	Bathroom	Landsize	Latitude	Longitude
count	6196.000000	6196.000000	6196.000000	6196.000000	6196.000000
mean	2.931407	1.576340	471.006940	-37.807904	144.990201
std	0.971079	0.711362	897.449881	0.075850	0.099165
min	1.000000	1.000000	0.000000	-38.164920	144.542370
25%	2.000000	1.000000	152.000000	-37.855438	144.926198
50%	3.000000	1.000000	373.000000	-37.802250	144.995800
75%	4.000000	2.000000	628.000000	-37.758200	145.052700
max	8.000000	8.000000	37000.000000	-37.457090	145.526350

```
X.head()
```

	Rooms	Bathroom	Landsize	Latitude	Longitude
1	2	1.0	156.0	-37.8079	144.9934
2	3	2.0	134.0	-37.8093	144.9944
4	4	1.0	120.0	-37.8072	144.9941
6	3	2.0	245.0	-37.8024	144.9993
7	2	1.0	256.0	-37.8060	144.9954

Das visuelle Überprüfen Ihrer Daten mit diesen Befehlen ist ein wichtiger Teil der Arbeit eines Data Scientists. Sie werden häufig Überraschungen im Datensatz finden, die einer genaueren Untersuchung bedürfen.

Building Your Model

Sie werden die **scikit-learn**-Bibliothek verwenden, um Ihre Modelle zu erstellen. Beim Codieren wird diese Bibliothek als **sklearn** geschrieben, wie Sie im Beispielcode sehen werden. Scikit-learn ist zweifellos die beliebteste Bibliothek für das Modellieren von Datentypen, die typischerweise in DataFrames gespeichert werden.

Die Schritte zum Erstellen und Verwenden eines Modells sind:

1. **Definieren:** Welcher Modelltyp wird es sein? Ein Entscheidungsbaum? Ein anderer Modelltyp? Auch andere Parameter des Modelltyps werden hier angegeben.
2. **Anpassen (Fit):** Muster aus den bereitgestellten Daten erfassen. Dies ist der Kern des Modellierens.
3. **Vorhersagen (Predict):** Genau das, was es klingt.
4. **Bewerten (Evaluate):** Bestimmen, wie genau die Vorhersagen des Modells sind.

Hier ist ein Beispiel, wie man ein Entscheidungsbaum-Modell mit scikit-learn definiert und es mit den Features und der Zielvariablen anpasst:

```
from sklearn.tree import DecisionTreeRegressor

# Define model. Specify a number for random_state to ensure same results each run
melbourne_model = DecisionTreeRegressor(random_state=1)

# Fit model
melbourne_model.fit(X, y)
```

DecisionTreeRegressor(random_state=1)

Viele Machine-Learning-Modelle erlauben eine gewisse Zufälligkeit beim Training des Modells. Das Festlegen einer Zahl für **random_state** stellt sicher, dass Sie bei jedem Durchlauf die gleichen Ergebnisse erhalten. Dies wird als gute Praxis angesehen. Sie können jede beliebige Zahl verwenden, und die Modellqualität wird nicht wesentlich davon abhängen, welche Zahl Sie wählen.

Wir haben jetzt ein angepasstes Modell, das wir für Vorhersagen verwenden können.

In der Praxis möchten Sie Vorhersagen für neue Häuser treffen, die auf den Markt kommen, anstatt für die Häuser, für die wir bereits Preise haben. Aber wir werden Vorhersagen für die ersten paar Zeilen der Trainingsdaten machen, um zu sehen, wie die **predict**-Funktion funktioniert.

```
print("Making predictions for the following 5 houses:")
print(X.head())
print("The predictions are")
print(melbourne_model.predict(X.head()))
```

Making predictions for the following 5 houses:

	Rooms	Bathroom	Landsize	Latitude	Longitude
1	2	1.0	156.0	-37.8079	144.9934
2	3	2.0	134.0	-37.8093	144.9944
4	4	1.0	120.0	-37.8072	144.9941
6	3	2.0	245.0	-37.8024	144.9993
7	2	1.0	256.0	-37.8060	144.9954

The predictions are

```
[1035000. 1465000. 1600000. 1876000. 1636000.]
```

Exercise: Your First Machine Learning

Recap

Bisher haben Sie Ihre Daten geladen und mit folgendem Code überprüft. Führen Sie diese Zelle aus, um Ihre Programmierungsumgebung dort einzurichten, wo der vorherige Schritt aufgehört hat.

```
# Code, den Sie zuvor verwendet haben, um Daten zu laden
import pandas as pd

# Pfad der Datei, die gelesen werden soll
iowa_file_path = '../input/home-data-for-ml-course/train.csv'

home_data = pd.read_csv(iowa_file_path)

# Code-Überprüfung einrichten
from learntools.core import binder
binder.bind(globals())
from learntools.machine_learning.ex3 import *

print("Setup Complete")
```

Setup Complete

Exercises, Step 1: Specify Prediction Target

Wählen Sie die Zielvariable aus, die dem Verkaufspreis entspricht. Speichern Sie diese in einer neuen Variablen namens **y**. Sie müssen eine Liste der Spalten ausdrucken, um den Namen der Spalte zu finden, die Sie benötigen.

```
# Liste der Spalten im Datensatz anzeigen
print(home_data.columns)

# Zielvariable (Verkaufspreis) auswählen und in y speichern
y = home_data['SalePrice']

# Check your answer
step_1.check()
```

```
Index(['Id', 'MSSubClass', 'MSZoning', 'LotFrontage', 'LotArea', 'Street',
      'Alley', 'LotShape', 'LandContour', 'Utilities', 'LotConfig',
      'LandSlope', 'Neighborhood', 'Condition1', 'Condition2', 'BldgType',
      'HouseStyle', 'OverallQual', 'OverallCond', 'YearBuilt', 'YearRemodAdd',
      'RoofStyle', 'RoofMatl', 'Exterior1st', 'Exterior2nd', 'MasVnrType',
      'MasVnrArea', 'ExterQual', 'ExterCond', 'Foundation', 'BsmtQual',
      'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinSF1',
      'BsmtFinType2', 'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF', 'Heating',
      'HeatingQC', 'CentralAir', 'Electrical', '1stFlrSF', '2ndFlrSF',
      'LowQualFinSF', 'GrLivArea', 'BsmtFullBath', 'BsmtHalfBath', 'FullBath',
      'HalfBath', 'BedroomAbvGr', 'KitchenAbvGr', 'KitchenQual',
      'TotRmsAbvGrd', 'Functional', 'Fireplaces', 'FireplaceQu', 'GarageType',
      'GarageYrBlt', 'GarageFinish', 'GarageCars', 'GarageArea', 'GarageQual',
      'GarageCond', 'PavedDrive', 'WoodDeckSF', 'OpenPorchSF',
      'EnclosedPorch', '3SsnPorch', 'ScreenPorch', 'PoolArea', 'PoolQC',
      'Fence', 'MiscFeature', 'MiscVal', 'MoSold', 'YrSold', 'SaleType',
      'SaleCondition', 'SalePrice'],
      dtype='object')
```

Correct

Step 2: Create X

Jetzt werden Sie ein DataFrame namens **X** erstellen, das die prädiktiven Features enthält.

Da Sie nur einige Spalten aus den Originaldaten verwenden möchten, erstellen Sie zunächst eine Liste mit den Namen der Spalten, die Sie in **X** haben möchten.

Verwenden Sie nur die folgenden Spalten in der Liste (Sie können die gesamte Liste kopieren und einfügen, um Tippaufwand zu sparen, müssen jedoch noch Anführungszeichen hinzufügen):

- LotArea
- YearBuilt
- 1stFlrSF
- 2ndFlrSF
- FullBath
- BedroomAbvGr
- TotRmsAbvGrd

Nachdem Sie diese Liste der Features erstellt haben, verwenden Sie sie, um das DataFrame zu erstellen, das Sie zum Anpassen des Modells verwenden werden.

```
# Liste der Features erstellen
features = ['LotArea', 'YearBuilt', '1stFlrSF', '2ndFlrSF', 'FullBath',
           'BedroomAbvGr', 'TotRmsAbvGrd']

# DataFrame X mit den ausgewählten Features erstellen
X = home_data[features]

step_2.check()
```

Correct

Review Data Bevor Sie ein Modell erstellen, werfen Sie einen schnellen Blick auf **X**, um zu überprüfen, ob es sinnvoll aussieht.

```
# Überprüfen Sie die ersten paar Zeilen von X
print(X.head())
# Die ersten paar Zeilen ausdrucken
print(X.describe())
```

	LotArea	YearBuilt	1stFlrSF	2ndFlrSF	FullBath	BedroomAbvGr	\
0	8450	2003	856	854	2	3	
1	9600	1976	1262	0	2	3	
2	11250	2001	920	866	2	3	
3	9550	1915	961	756	1	3	
4	14260	2000	1145	1053	2	4	

	TotRmsAbvGrd
0	8
1	6
2	6
3	7
4	9

	LotArea	YearBuilt	1stFlrSF	2ndFlrSF	FullBath
count	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000
mean	10516.828082	1971.267808	1162.626712	346.992466	1.565068
std	9981.264932	30.202904	386.587738	436.528436	0.550916
min	1300.000000	1872.000000	334.000000	0.000000	0.000000
25%	7553.500000	1954.000000	882.000000	0.000000	1.000000
50%	9478.500000	1973.000000	1087.000000	0.000000	2.000000
75%	11601.500000	2000.000000	1391.250000	728.000000	2.000000
max	215245.000000	2010.000000	4692.000000	2065.000000	3.000000

	BedroomAbvGr	TotRmsAbvGrd
count	1460.000000	1460.000000
mean	2.866438	6.517808
std	0.815778	1.625393
min	0.000000	2.000000
25%	2.000000	5.000000
50%	3.000000	6.000000
75%	3.000000	7.000000
max	8.000000	14.000000

Step 3: Specify and Fit Model

Create a DecisionTreeRegressor and save it iowa_model. Ensure you've done the relevant import from sklearn to run this command.

Then fit the model you just created using the data in X and y that you saved above.

```
# Relevante Imports aus sklearn
from sklearn.tree import DecisionTreeRegressor

# Erstellen des DecisionTreeRegressor-Modells
iowa_model = DecisionTreeRegressor(random_state=1)

# Modell anpassen
iowa_model.fit(X, y)
```

Correct

Step 4: Make Predictions

Machen Sie Vorhersagen mit dem Befehl **predict** des Modells unter Verwendung von **X** als Daten. Speichern Sie die Ergebnisse in einer Variablen namens **predictions**.

```
# Vorhersagen mit dem Modell machen
predictions = iowa_model.predict(X)

print(predictions)

# Check your answer
step_4.check()
```

Correct

Teil 1 Lessons 1-3:

1) Begriffsklärung

- **Prediction:** Eine Vorhersage, z.B. des Hauspreises, die ein Modell auf Basis von Eingabedaten trifft.
- **Pattern:** Ein Muster oder eine Regelmäßigkeit in den Daten, die vom Modell erkannt wird.
- **Fitting:** Der Prozess, bei dem das Modell die Muster in den Trainingsdaten lernt (Modellanpassung).
- **Training:** Der Vorgang, bei dem das Modell mit bekannten Daten (Trainingsdaten) "gefüttert" wird, um Vorhersagen treffen zu können.

2) Was ist ein Decision Tree und wie können seine Predictions verbessert werden?

Ein **Decision Tree** ist ein Modell, das Entscheidungen anhand von Merkmalen trifft, indem es Daten in Gruppen mit ähnlichen Ergebnissen aufteilt. Die Vorhersage erfolgt durch das Folgen eines Pfades im Baum bis zu einem Blatt (Leaf), wo die Vorhersage steht.

Verbesserung: Durch tiefere Bäume mit mehr "Splits" können mehr Einflussfaktoren berücksichtigt werden (z. B. mehr Features wie Badezimmer, Grundstücksgröße, etc.).

3) Was ist ein DataFrame?

Ein **DataFrame** ist eine tabellenartige Datenstruktur in Pandas, vergleichbar mit einem Excel-Blatt oder einer SQL-Tabelle. Sie enthält Zeilen und Spalten, wobei jede Spalte ein Attribut darstellt.

4) Mit welcher Pandas-Funktion können Sie eine erste Analyse der Daten vornehmen?

```
melbourne_data.describe()
```

5) Was bedeuten count, mean, std, min, 25%, 50%, 75% und max?

- count: Anzahl der vorhandenen (nicht fehlenden) Werte
- mean: Durchschnittswert
- std: Standardabweichung (Maß für Streuung)
- min: Kleinster Wert
- 25%: 25. Perzentil – ein Viertel der Werte liegen darunter
- 50%: Median – die Hälfte der Werte liegen darunter
- 75%: 75. Perzentil – drei Viertel der Werte liegen darunter
- max: Größter Wert

6) Iowa-Daten: Was ist die durchschnittliche Grundstücksfläche und wie alt ist das neueste Haus?

Im Beispieltext beziehen sich die Werte auf die Melbourne-Daten:

- Durchschnittliche Grundstücksfläche: 471.01 m²
- Neuester Hausbau: 2018

(Die Iowa-Daten werden in einem späteren Teil verwendet und haben laut Text keine fehlenden Werte.)

7) Wie können Sie eine Liste aller Attribute anzeigen?

```
melbourne_data.columns
```

8) Was wird üblicherweise mit "y" in einem MLM bezeichnet?

Mit "y" wird der Prediction Target bezeichnet – also die Zielvariable, die vorhergesagt werden soll (z. B. der Hauspreis).

9) Was mit "X"?

"X" bezeichnet die Features, also die Eingabevariablen oder Merkmale, die verwendet werden, um "y" vorherzusagen.

10) Was ist der Unterschied zwischen dem Prediction Target und den Features?

- Prediction Target (y): Die Variable, die vorhergesagt werden soll.
- Features (X): Die Variablen, die dem Modell als Input dienen, um die Vorhersage für "y" zu treffen.

11) Code um ein Modell anhand von X und y zu bauen

```
from sklearn.tree import DecisionTreeRegressor

# Modell definieren
melbourne_model = DecisionTreeRegressor(random_state=1)

# Modell fitten (trainieren)
melbourne_model.fit(X, y)

# Vorhersage für neue Daten (optional)
predictions = melbourne_model.predict(X.head())
print(predictions)
```

4. Model Validation

► What is Model Validation

Ein häufiger Fehler besteht darin, die Vorhersagen des Modells mit den Trainingsdaten zu vergleichen. Das ist problematisch, da das Modell diese Daten bereits kennt und die Bewertung dadurch verzerrt ist.

Stattdessen sollte man die Modellqualität mit einer einfachen Kennzahl zusammenfassen, da es wenig sinnvoll ist, Tausende von Vorhersagen manuell zu vergleichen.

Eine gängige Metrik dafür ist der mittlere absolute Fehler (engl. Mean Absolute Error, MAE). Dabei wird:

- für jede Vorhersage der Fehler berechnet: $\text{Fehler} = \text{tatsächlicher Wert} - \text{vorhergesagter Wert}$
 - dann wird der absolute Wert jedes Fehlers genommen (also ohne Vorzeichen),
 - und schließlich der Durchschnitt aller Fehler berechnet.
- MAE bedeutet also: Im Durchschnitt liegen die Vorhersagen um etwa X vom tatsächlichen Wert daneben.

```
# Data Loading Code Hidden Here
import pandas as pd

# Load data
melbourne_file_path = '../input/melbourne-housing-snapshot/melb_data.csv'
melbourne_data = pd.read_csv(melbourne_file_path)
# Filter rows with missing price values
filtered_melbourne_data = melbourne_data.dropna(axis=0)
```

```
# Choose target and features
y = filtered_melbourne_data.Price
melbourne_features = ['Rooms', 'Bathroom', 'Landsize', 'BuildingArea',
                      'YearBuilt', 'Lattitude', 'Longtitude']
X = filtered_melbourne_data[melbourne_features]

from sklearn.tree import DecisionTreeRegressor
# Define model
melbourne_model = DecisionTreeRegressor()
# Fit model
melbourne_model.fit(X, y)
```

DecisionTreeRegressor()

Sobald wir ein Modell haben, berechnen wir den mittleren absoluten Fehler wie folgt:

```
from sklearn.metrics import mean_absolute_error

predicted_home_prices = melbourne_model.predict(X)
mean_absolute_error(y, predicted_home_prices)
```

434.71594577146544

► The Problem with "In-Sample" Scores

Die zuvor berechnete Modellbewertung nennt man „In-Sample“-Score, weil für Training und Bewertung dieselben Daten verwendet wurden. Das ist problematisch.

Ein Beispiel:

Stell dir vor, die Türfarbe hat in Wirklichkeit keinen Einfluss auf den Hauspreis. Aber zufällig sind in deinem Trainingsdatensatz alle Häuser mit grüner Tür sehr teuer. Das Modell erkennt dieses Muster und lernt, dass grüne Türen teure Häuser bedeuten.

Im Trainingsdatensatz funktioniert diese Vorhersage gut – weil das Modell genau diesen Datensatz gesehen hat.

In der Realität (mit neuen Daten) gilt dieses Muster aber vielleicht nicht, und die Vorhersagen des Modells wären dann falsch und unzuverlässig.

Fazit: Ein Modell sollte auf neuen, unbekannten Daten getestet werden – nicht auf den Daten, mit denen es trainiert wurde. Dafür verwendet man sogenannte Validierungsdaten: ein Teil der Daten, der nicht zum Training verwendet wurde, sondern nur zur Bewertung der Modellqualität.

► Coding It

Die Bibliothek scikit-learn stellt die Funktion `train_test_split` bereit, um die Daten in zwei Teile zu trennen:

- Trainingsdaten – um das Modell zu erstellen
- Validierungsdaten – um die Modellgenauigkeit zu bewerten (z. B. mit dem MAE)

Hier ist der Python-Code dazu:

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error

# Aufteilen der Merkmale (X) und Zielwerte (y) in Trainings- und Validierungsdaten
train_X, val_X, train_y, val_y = train_test_split(X, y, random_state=0)

# Modell definieren
melbourne_model = DecisionTreeRegressor()

# Modell mit Trainingsdaten trainieren
melbourne_model.fit(train_X, train_y)

# Vorhersagen für die Validierungsdaten machen
val_predictions = melbourne_model.predict(val_X)

# MAE berechnen und ausgeben
print(mean_absolute_error(val_y, val_predictions))
```

265806.91478373145

► Wow!

- Auf neuen, unbekannten Daten (Out-of-Sample) liegt der Fehler jedoch bei über 250.000 Dollar.
- Das zeigt den riesigen Unterschied zwischen einem Modell, das auf bekannte Daten „perfekt“ passt, und einem, das in der Praxis kaum brauchbar ist.

Zum Vergleich: Der durchschnittliche Hauswert in den Validierungsdaten beträgt 1,1 Millionen Dollar. Ein Fehler von 250.000 Dollar entspricht also etwa einem Viertel des tatsächlichen Werts – das ist viel zu hoch.

► Exercise

Recap

- Du hast ein Modell erstellt.
- Jetzt wirst du testen, wie gut dein Modell wirklich ist.
- Im nächsten Schritt wird das Programmier-Umfeld vorbereitet, damit du direkt dort weitermachen kannst, wo du im vorherigen Abschnitt aufgehört hast.

- Dazu musst du einfach die nächste Code-Zelle ausführen, die alles Nötige lädt (z. B. Daten, Bibliotheken, das Modell usw.).

```
# Code you have previously used to load data
import pandas as pd
from sklearn.tree import DecisionTreeRegressor

# Path of the file to read
iowa_file_path = '../input/home-data-for-ml-course/train.csv'

home_data = pd.read_csv(iowa_file_path)
y = home_data.SalePrice
feature_columns = ['LotArea', 'YearBuilt', '1stFlrSF', '2ndFlrSF', 'FullBath',
'BedroomAbvGr', 'TotRmsAbvGrd']
X = home_data[feature_columns]

# Specify Model
iowa_model = DecisionTreeRegressor()
# Fit Model
iowa_model.fit(X, y)

print("First in-sample predictions:", iowa_model.predict(X.head()))
print("Actual target values for those homes:", y.head().tolist())

# Set up code checking
from learntools.core import binder
binder.bind(globals())
from learntools.machine_learning.ex4 import *
print("Setup Complete")
```

```
First in-sample predictions: [208500. 181500. 223500. 140000. 250000.]
Actual target values for those homes: [208500, 181500, 223500, 140000, 250000]
Setup Complete
```

Step 1: Split Your Data

```
# Import the train_test_split function and uncomment
from sklearn.model_selection import train_test_split

# fill in and uncomment
train_X, val_X, train_y, val_y = train_test_split(X, y, random_state=1)

# Check your answer
step_1.check()
```

Correct

Erklärung:

- `train_test_split(X, y, random_state=1)` teilt die Merkmale X und das Ziel y in Trainings- und Validierungsdaten.
- `random_state=1` sorgt dafür, dass die Aufteilung jedes Mal gleich ist – wichtig für automatische Tests.

Step 2: Specify and Fit the Model

```
# Specify the model
iowa_model = DecisionTreeRegressor(random_state=1)

# Fit iowa_model with the training data
iowa_model.fit(train_X, train_y)

# Check your answer
step_2.check()
```

```
[186500. 184000. 130000.  92000. 164500. 220000. 335000. 144152. 215000.
 262000.]
[186500. 184000. 130000.  92000. 164500. 220000. 335000. 144152. 215000.
 262000.]
```

Correct

Erklärung:

- `DecisionTreeRegressor(random_state=1)` erstellt das Modell mit einer festen Zufallskomponente – wichtig für Reproduzierbarkeit.
- `.fit(train_X, train_y)` trainiert das Modell mit den Trainingsdaten.

Step 3: Make Predictions with Validation data

```
# Predict with all validation observations
val_predictions = iowa_model.predict(val_X)

# Check your answer
step_3.check()

# print the top few validation predictions
print(val_predictions[:5])

# print the top few actual prices from validation data
print(val_y[:5])
```

Correct

```
[186500. 184000. 130000.  92000. 164500.]
258      231500
267      179500
288      122000
649       84500
1233     142000
Name: SalePrice, dtype: int64
```

Erklärung:

- `iowa_model.predict(val_X)` erstellt Vorhersagen auf Basis der Validierungsmerkmale.
- Mit `[:5]` gibst du die ersten 5 Vorhersagen bzw. echten Werte aus – das reicht, um einen schnellen Eindruck zu bekommen.

Step 4: Calculate the Mean Absolute Error in Validation Data:

```
from sklearn.metrics import mean_absolute_error

# Berechne den MAE zwischen den Vorhersagen und den tatsächlichen Werten
val_mae = mean_absolute_error(val_y, val_predictions)

# Uncomment following line to see the validation_mae
print(val_mae)

# Check your answer
step_4.check()
```

29652.931506849316

Correct

Erklärung:

- `mean_absolute_error(val_y, val_predictions)` berechnet den mittleren absoluten Fehler zwischen den tatsächlichen (`val_y`) und den vorhergesagten Werten (`val_predictions`).
- Die `print(val_mae)` gibt den berechneten MAE aus, damit du den Fehler sehen kannst.

Teil 2 Lesson 4: Model Validation

1) Welche einzelne Metrik wird oft zur Analyse der Modellqualität verwendet?

Die **Mean Absolute Error (MAE)** ist eine häufig verwendete Metrik zur Analyse der Modellqualität. Sie gibt an, wie stark die Vorhersagen des Modells im Durchschnitt von den tatsächlichen Werten abweichen.

2) Wie berechnet man den MAE?

Der MAE wird wie folgt berechnet:

1. Für jede Vorhersage wird die Differenz zwischen dem tatsächlichen Wert und dem vorhergesagten Wert berechnet.
2. Diese Fehler werden in den absoluten Wert umgewandelt (also alle positiv gemacht).
3. Anschließend wird der Durchschnitt dieser absoluten Fehler berechnet.

Formel:

```
from sklearn.metrics import mean_absolute_error

mean_absolute_error(y_true, y_pred)
```

3) Was ist ein "In-Sample" Score und warum ist dieser alles andere als aussagekräftig?

Ein "In-Sample" Score ist ein Maß für die Modellqualität, das auf denselben Daten berechnet wird, die auch zum Trainieren des Modells verwendet wurden. Problem: Das Modell kann sich an spezielle Muster in den Trainingsdaten "anpassen" (Overfitting), die in neuen Daten nicht existieren. Dadurch erscheint das Modell besser als es tatsächlich ist, wenn es auf unbekannte Daten trifft.

4) Wie kann man eine sinnvolle und aussagekräftige Validierung eines Modells vornehmen?

Man trennt die Daten in zwei Teile:

- Einen Trainingsdatensatz, um das Modell zu trainieren.
- Einen Validierungsdatensatz, um die Qualität des Modells auf Daten zu testen, die es noch nicht gesehen hat.

Dies verhindert, dass das Modell zu sehr auf die Trainingsdaten abgestimmt wird, und gibt einen realistischeren Eindruck von der Leistungsfähigkeit.

5) Was macht die Funktion `train_test_split`?

Die Funktion `train_test_split` aus `sklearn.model_selection`:

- Teilt den Datensatz zufällig in zwei Teile auf:
 - Trainingsdaten (`train_X`, `train_y`)
 - Validierungsdaten (`val_X`, `val_y`)
- Mit dem Parameter `random_state` kann man sicherstellen, dass bei jedem Durchlauf die gleiche Aufteilung erfolgt.

6) Code der Iowa-Immobilien von Step 1 bis Step 4 mit Erklärung

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error

# Step 1: Split Your Data
train_X, val_X, train_y, val_y = train_test_split(X, y, random_state=1)

# Step 2: Define the Model
iowa_model = DecisionTreeRegressor(random_state=1)

# Step 3: Fit Model
iowa_model.fit(train_X, train_y)

# Step 4: Calculate the MAE in Validation Data
val_predictions = iowa_model.predict(val_X)
val_mae = mean_absolute_error(val_y, val_predictions)
print("Validation MAE:", val_mae)
```

Erklärung:

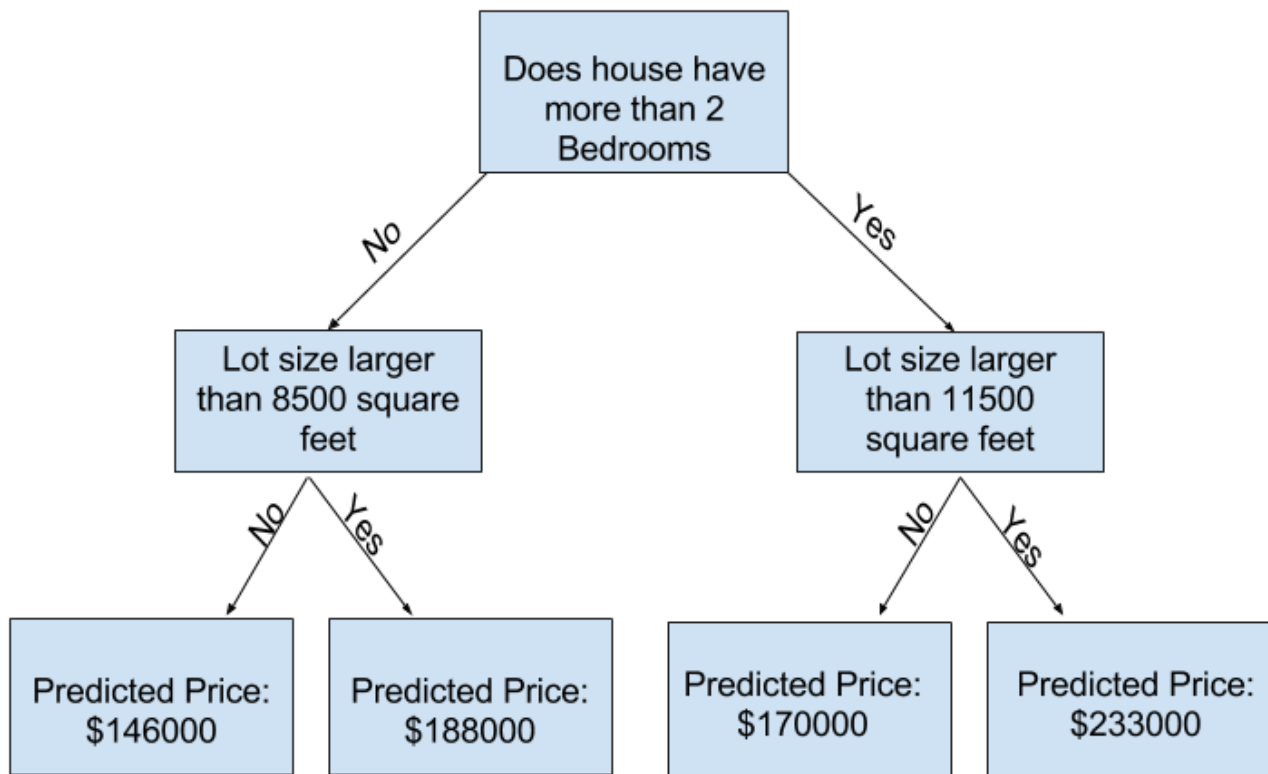
- `train_test_split(...)`: Teilt die Merkmale (X) und Zielwerte (y) in Trainings- und Validierungsdaten.
- `DecisionTreeRegressor(...)`: Erstellt ein Entscheidungsbaum-Modell.
- `fit(...)`: Trainiert das Modell mit den Trainingsdaten.
- `predict(...)`: Macht Vorhersagen mit den Validierungsdaten.
- `mean_absolute_error(...)`: Berechnet die mittlere absolute Abweichung zwischen den tatsächlichen und vorhergesagten Werten im Validierungsdatensatz.

5. Underfitting and Overfitting

► Experimenting With Different Models

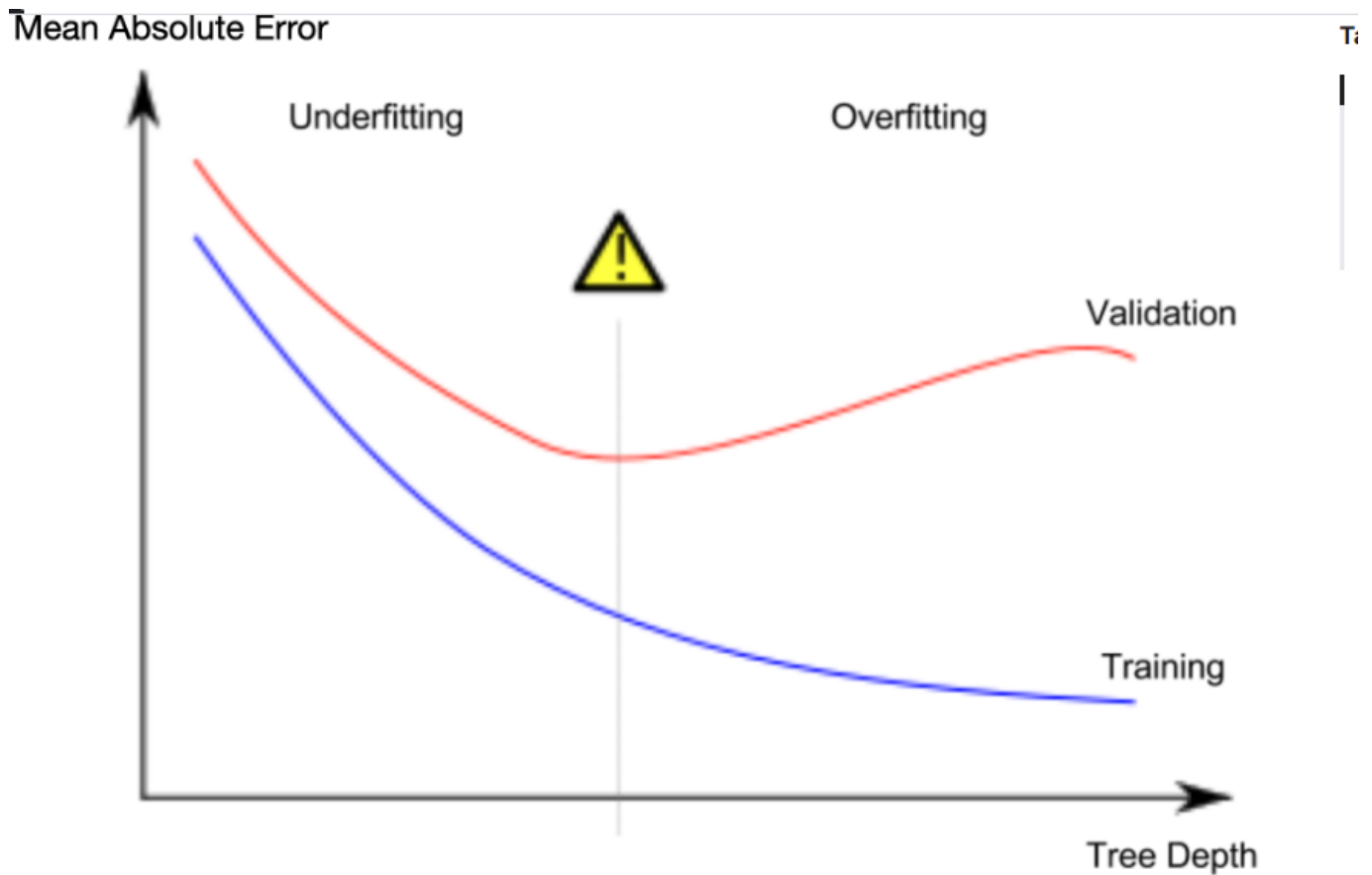
Wichtige Überlegungen zur Baumtiefe:

- Der Entscheidungsbaum hat viele Parameter, aber einer der wichtigsten ist die Baumtiefe.
- Baumtiefe ist ein Maß dafür, wie viele Teilsplits der Baum macht, bevor er eine Vorhersage trifft. Eine flache Baumtiefe bedeutet weniger Teilsplits. Beispiel:
 - Bei einer Baumtiefe von 1 gibt es nur 2 Gruppen von Häusern.
 - Mit Baumtiefe von 2 gibt es 4 Gruppen.
 - Bei einer Baumtiefe von 10 gibt es 1024 Gruppen (so viele „Blätter“ des Baums).



Overfitting vs. Underfitting:

- Overfitting (Überanpassung):
 - Ein sehr tiefer Baum wird die Daten zu genau lernen. Er macht sehr präzise Vorhersagen für das Training, aber ist unzuverlässig bei neuen, unbekannten Daten, weil er die Unterschiede innerhalb des Trainingsdatensatzes zu sehr ausnutzt. Das heißt, er verallgemeinert schlecht.
- Underfitting (Unteranpassung):
 - Visuell suchst du nach dem „Tiefpunkt“ der Validierungsgenauigkeit, um die beste Modellkomplexität zu finden – also den Punkt, an dem der Baum genug Komplexität hat, um relevante Muster zu erfassen, aber nicht so komplex wird, dass er nur die Trainingsdaten nachahmt.

**Ziel:**

- Du möchtest das „Goldene Mittel“ finden, also eine Baumtiefe, die weder zu flach noch zu tief ist.
- Visuell suchst du nach dem „Tiefpunkt“ der Validierungsgenauigkeit, um die beste Modellkomplexität zu finden – also den Punkt, an dem der Baum genug Komplexität hat, um relevante Muster zu erfassen, aber nicht so komplex wird, dass er nur die Trainingsdaten nachahmt.

► Example

Es gibt mehrere Möglichkeiten, die Tiefe eines Entscheidungsbaums zu steuern. Ein hilfreicher Parameter ist `max_leaf_nodes`, der steuert, wie viele Blätter der Baum insgesamt haben kann. Je mehr Blätter wir erlauben, desto mehr bewegen wir uns von der Unteranpassung (underfitting) hin zur Überanpassung (overfitting).

Die Funktion `get_mae` hilft uns dabei, den MAE für verschiedene Werte von `max_leaf_nodes` zu vergleichen.

Code:

```
from sklearn.metrics import mean_absolute_error
from sklearn.tree import DecisionTreeRegressor
import pandas as pd

# Hilfsfunktion, um MAE mit unterschiedlicher Anzahl von max_leaf_nodes zu berechnen
def get_mae(max_leaf_nodes, train_X, val_X, train_y, val_y):
    model = DecisionTreeRegressor(max_leaf_nodes=max_leaf_nodes, random_state=0)
    model.fit(train_X, train_y)
    preds_val = model.predict(val_X)
    mae = mean_absolute_error(val_y, preds_val)
    return(mae)
```

```
# Daten laden
melbourne_file_path = '../input/melbourne-housing-snapshot/melb_data.csv'
melbourne_data = pd.read_csv(melbourne_file_path)

# Fehlende Werte entfernen
filtered_melbourne_data = melbourne_data.dropna(axis=0)

# Zielvariable und Merkmale auswählen
y = filtered_melbourne_data.Price
melbourne_features = ['Rooms', 'Bathroom', 'Landsize', 'BuildingArea',
                      'YearBuilt', 'Lattitude', 'Longtitude']
X = filtered_melbourne_data[melbourne_features]

from sklearn.model_selection import train_test_split

# Daten in Trainings- und Validierungsdaten aufteilen
train_X, val_X, train_y, val_y = train_test_split(X, y, random_state=0)

# Vergleich der MAE mit unterschiedlichen max_leaf_nodes-Werten
for max_leaf_nodes in [5, 50, 500, 5000]:
    my_mae = get_mae(max_leaf_nodes, train_X, val_X, train_y, val_y)
    print("Max leaf nodes: %d \t\t Mean Absolute Error: %d" %(max_leaf_nodes,
my_mae))
```

Erklärung

1. **get_mae**: Diese Funktion erstellt ein Modell, trainiert es und berechnet den MAE.
2. **max_leaf_nodes**: Dieser Parameter steuert, wie viele Blätter der Entscheidungsbaum haben kann. Durch das Ändern dieses Werts kannst du sehen, wie sich die Anzahl der Blätter auf die Modellgenauigkeit auswirkt.
3. **Daten laden**: Der Datensatz wird eingelesen und für das Modell vorbereitet.
4. **MAE-Vergleich**: Ein for-loop wird verwendet, um Modelle mit verschiedenen Werten für max_leaf_nodes zu testen und die Ergebnisse zu vergleichen.

Was du hier lernst

- Was du hier lernst: Durch die Variation der Baumtiefe (mit max_leaf_nodes) kannst du beobachten, wie sich die Modellgenauigkeit verändert. Du suchst den Punkt, an dem der MAE am geringsten ist, ohne das Modell zu überanpassen.

► Exercise

Recap

Sie haben Ihr erstes Modell erstellt. Nun ist es an der Zeit, die Größe des Baums zu optimieren, um bessere Vorhersagen zu treffen. Führen Sie diese Zelle aus, um Ihre Programmierumgebung dort einzurichten, wo der vorherige Schritt aufgehört hat.

```
# Code you have previously used to load data
import pandas as pd
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor

# Path of the file to read
iowa_file_path = '../input/home-data-for-ml-course/train.csv'

home_data = pd.read_csv(iowa_file_path)
# Create target object and call it y
y = home_data.SalePrice
# Create X
features = ['LotArea', 'YearBuilt', '1stFlrSF', '2ndFlrSF', 'FullBath',
            'BedroomAbvGr', 'TotRmsAbvGrd']
X = home_data[features]

# Split into validation and training data
train_X, val_X, train_y, val_y = train_test_split(X, y, random_state=1)

# Specify Model
iowa_model = DecisionTreeRegressor(random_state=1)
# Fit Model
iowa_model.fit(train_X, train_y)

# Make validation predictions and calculate mean absolute error
val_predictions = iowa_model.predict(val_X)
val_mae = mean_absolute_error(val_predictions, val_y)
print("Validation MAE: {:.0f}".format(val_mae))

# Set up code checking
from learntools.core import binder
binder.bind(globals())
from learntools.machine_learning.ex5 import *
print("\nSetup complete")
```

Validation MAE: 29,653

Setup complete

Exercises

Sie könnten die Funktion `get_mae` selbst schreiben. Wir stellen sie Ihnen vorerst zur Verfügung. Es handelt sich um dieselbe Funktion, die Sie in der vorherigen Lektion gelesen haben. Führen Sie einfach die folgende Zelle aus.

```
def get_mae(max_leaf_nodes, train_X, val_X, train_y, val_y):  
    model = DecisionTreeRegressor(max_leaf_nodes=max_leaf_nodes, random_state=0)  
    model.fit(train_X, train_y)  
    preds_val = model.predict(val_X)  
    mae = mean_absolute_error(val_y, preds_val)  
    return(mae)
```

Step 1: Compare Different Tree Sizes

```
# Kandidatenwerte für max_leaf_nodes  
candidate_max_leaf_nodes = [5, 25, 50, 100, 250, 500]  
  
# Variable zum Speichern des besten MAE und der besten Baumgröße  
best_mae = float('inf') # Startwert für den besten MAE (unendlich hoch)  
best_tree_size = None # Variable für die beste Baumgröße  
  
# Schleife, um den MAE für verschiedene max_leaf_nodes zu berechnen  
for max_leaf_nodes in candidate_max_leaf_nodes:  
    # Berechne den MAE für den aktuellen Wert von max_leaf_nodes  
    current_mae = get_mae(max_leaf_nodes, train_X, val_X, train_y, val_y)  
  
    # Überprüfen, ob der aktuelle MAE besser (kleiner) ist  
    if current_mae < best_mae:  
        best_mae = current_mae  
        best_tree_size = max_leaf_nodes  
  
# Ausgabe des besten max_leaf_nodes  
print("Best max_leaf_nodes:", best_tree_size)
```

Best max_leaf_nodes: 100

Erklärung:

1. Kandidatenwerte für `max_leaf_nodes`: Wir legen die möglichen Werte in der Liste `candidate_max_leaf_nodes` fest.
2. Schleife: Für jedes Element in `candidate_max_leaf_nodes` wird der MAE mit der Funktion `get_mae` berechnet.
3. Vergleich: Wir vergleichen den aktuellen MAE mit dem besten MAE (`best_mae`). Wenn der aktuelle MAE besser ist (kleiner), speichern wir den aktuellen Wert von `max_leaf_nodes` in `best_tree_size`.
4. Ergebnis: Am Ende der Schleife haben wir den Wert von `max_leaf_nodes`, der den besten (kleinsten) MAE liefert.

Step 2: Fit Model Using All Data

```
# Erstelle das finale Modell mit dem besten Wert für max_leaf_nodes
```

```
final_model = DecisionTreeRegressor(max_leaf_nodes=best_tree_size, random_state=1)

# Trainiere das finale Modell mit allen verfügbaren Daten (X, y)
final_model.fit(X, y)
```

Erklärung:

- `max_leaf_nodes=best_tree_size`: Wir verwenden den besten Wert für `max_leaf_nodes`, den wir aus der vorherigen Schleife ermittelt haben.
- `fit(X, y)`: Anstatt das Modell nur mit den Trainingsdaten zu trainieren, verwenden wir nun den gesamten Datensatz (`X` und `y`), um das Modell weiter zu optimieren. Dies ist eine gängige Praxis, wenn man das Modell nach der Auswahl des besten Modells vollständig trainieren möchte.

6. Random Forests

► Introductoin & Example

Einführung: Entscheidungsbäume haben einen großen Nachteil: Ein tiefer Baum mit vielen Blättern neigt zu Überanpassung (Overfitting), da jede Vorhersage nur auf einer kleinen Menge historischer Daten basiert. Ein flacher Baum hingegen hat oft Unteranpassung (Underfitting), da er nicht genug Details aus den Rohdaten erfasst.

Trotz dieser Probleme verwenden viele fortschrittliche Modelle wie der Random Forest kluge Ideen, um diese Tendenzen zu überwinden. Der Random Forest verwendet viele Entscheidungsbäume und trifft eine Vorhersage, indem er die Vorhersagen jedes Baums durchschnittlich. Dieses Modell hat in der Regel eine viel bessere Vorhersagegenauigkeit als ein einzelner Entscheidungsbaum und funktioniert in den meisten Fällen gut mit den Standardparametern.

Beispiel: Der Code zum Laden der Daten wurde bereits mehrfach gezeigt. Am Ende des Ladeprozesses haben wir die folgenden Variablen:

- `train_X`: (Trainingseingaben)
- `val_X`: (Validierungseingaben)
- `train_y`: (Trainingsziel)
- `val_y`: (Validierungsziel)

Dann verwenden wir den `RandomForestRegressor` aus der `sklearn`-Bibliothek, um das Modell zu erstellen und zu trainieren. Anschließend wird der Fehler auf den Validierungsdaten mit dem Mean Absolute Error (MAE) berechnet:

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error

# Random Forest Modell erstellen
forest_model = RandomForestRegressor(random_state=1)

# Modell trainieren
forest_model.fit(train_X, train_y)
```

```
# Vorhersagen auf den Validierungsdaten machen
melb_preds = forest_model.predict(val_X)

# MAE berechnen und ausgeben
print(mean_absolute_error(val_y, melb_preds))
```

191669.7536453626

► Exercise

Recap

```
# Code you have previously used to load data
import pandas as pd
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor

# Path of the file to read
iowa_file_path = '../input/home-data-for-ml-course/train.csv'

home_data = pd.read_csv(iowa_file_path)
# Create target object and call it y
y = home_data.SalePrice
# Create X
features = ['LotArea', 'YearBuilt', '1stFlrSF', '2ndFlrSF', 'FullBath',
            'BedroomAbvGr', 'TotRmsAbvGrd']
X = home_data[features]

# Split into validation and training data
train_X, val_X, train_y, val_y = train_test_split(X, y, random_state=1)

# Specify Model
iowa_model = DecisionTreeRegressor(random_state=1)
# Fit Model
iowa_model.fit(train_X, train_y)

# Make validation predictions and calculate mean absolute error
val_predictions = iowa_model.predict(val_X)
val_mae = mean_absolute_error(val_predictions, val_y)
print("Validation MAE when not specifying max_leaf_nodes:
{:,.0f}".format(val_mae))

# Using best value for max_leaf_nodes
iowa_model = DecisionTreeRegressor(max_leaf_nodes=100, random_state=1)
iowa_model.fit(train_X, train_y)
val_predictions = iowa_model.predict(val_X)
val_mae = mean_absolute_error(val_predictions, val_y)
print("Validation MAE for best value of max_leaf_nodes: {:,.0f}".format(val_mae))
```

```
# Set up code checking
from learntools.core import binder
binder.bind(globals())
from learntools.machine_learning.ex6 import *
print("\nSetup complete")
```

Validation MAE when not specifying max_leaf_nodes: 29,653

Validation MAE for best value of max_leaf_nodes: 27,283

Setup complete

Step 1: Use a Random Forest

```
from sklearn.ensemble import RandomForestRegressor

# 1. Modell definieren
rf_model = RandomForestRegressor(random_state=1)

# 2. Modell trainieren
rf_model.fit(train_X, train_y)

# 3. MAE auf den Validierungsdaten berechnen
rf_val_mae = mean_absolute_error(val_y, rf_model.predict(val_X))

# Ausgabe des MAE
print("Validation MAE for Random Forest Model: {}".format(rf_val_mae))

step_1.check()
```

Validation MAE for Random Forest Model: 21857.15912981083

Correct

Erklärung

1. Modell definieren: Wir erstellen das Modell `rf_model` mit `RandomForestRegressor(random_state=1)`, wobei `random_state=1` sicherstellt, dass die Ergebnisse bei wiederholtem Ausführen des Codes konsistent sind.
2. Modell trainieren: Mit `rf_model.fit(train_X, train_y)` trainieren wir das Modell mit den Trainingsdaten.
3. MAE berechnen: Mit `mean_absolute_error(val_y, rf_model.predict(val_X))` berechnen wir den MAE des Modells, indem wir die Vorhersagen für `val_X` machen und diese mit den tatsächlichen Werten `val_y` vergleichen.

1	How Models Work The first step if you're new to machine learning.	✓	
2	Basic Data Exploration Load and understand your data.	✓	✓
3	Your First Machine Learning Model Building your first model. Hurray!	✓	✓
4	Model Validation Measure the performance of your model, so you can test and compare alternatives.	✓	✓
5	Underfitting and Overfitting Fine-tune your model for better performance.	✓	⏮
6	Random Forests Using a more sophisticated machine learning algorithm.	✓	✓

Modellgüte: Overfitting, Underfitting & Random Forests

1) Was versteht man unter Overfitting und Underfitting eines Modells?

- **Overfitting (Überanpassung)** tritt auf, wenn ein Modell sich zu sehr an die Trainingsdaten anpasst. Es erkennt auch zufällige Muster oder Ausreißer, die nicht generalisierbar sind. Das Modell hat in den Trainingsdaten sehr geringe Fehler, aber auf neuen (validierten) Daten sind die Vorhersagen oft schlecht.
- **Underfitting (Unteranpassung)** bedeutet, dass das Modell zu einfach ist und die tatsächlichen Muster in den Daten nicht ausreichend erfasst. Es hat sowohl im Training als auch in der Validierung eine hohe Fehlerquote.

2) Wie kann man die Modellqualität garantieren, also Overfitting und Underfitting verhindern?

- **Verwendung von Validierungsdaten:** Man teilt die Daten in Trainings- und Validierungsdaten (z.B. mit `train_test_split`). So testet man die Modellqualität auf neuen, ungesehenen Daten.
- **Modellkomplexität steuern:** Bei Entscheidungsbäumen z. B. über Parameter wie `max_leaf_nodes`. Eine zu hohe Anzahl führt zu Overfitting, eine zu geringe zu Underfitting. Ziel ist es, die optimale Komplexität zu finden.
- **Vergleich verschiedener Modelle und Parameter:** Man testet systematisch verschiedene Einstellungen und wählt das Modell mit dem besten Ergebnis auf den Validierungsdaten.

Beispielhafter Vergleich:

```
def get_mae(max_leaf_nodes, train_X, val_X, train_y, val_y):
    model = DecisionTreeRegressor(max_leaf_nodes=max_leaf_nodes, random_state=0)
    model.fit(train_X, train_y)
    preds_val = model.predict(val_X)
    mae = mean_absolute_error(val_y, preds_val)
    return(mae)

for max_leaf_nodes in [5, 50, 500, 5000]:
    my_mae = get_mae(max_leaf_nodes, train_X, val_X, train_y, val_y)
    print("Max leaf nodes:", max_leaf_nodes, "\t MAE:", my_mae)
```

3) Was sind Random Forests? Was ist daran „zufällig“?

- Random Forests sind Ensemble-Modelle, die viele Entscheidungsbäume bauen (oft Hunderte) und deren Vorhersagen mitteln.
- Zufälligkeit entsteht durch zwei Dinge:
 1. Jeder Baum bekommt nur einen Zufallsanteil der Trainingsdaten (sog. Bootstrap-Sampling).
 2. Bei jedem Split im Baum wird nur ein zufälliger Teil der Merkmale betrachtet, nicht alle.

Dadurch sind die einzelnen Bäume unterschiedlich, und das Mittel ihrer Vorhersagen ist robuster und genauer.

4) Welche Probleme können Random Forests besonders gut lösen?

- Reduktion von Overfitting: Einzelne Bäume können stark overfitten, aber das Mittel vieler zufälliger Bäume gleicht Ausreißer aus.
- Gute Standardleistung: Auch ohne viel Feintuning erreichen Random Forests oft schon sehr gute Ergebnisse.
- Stabilität: Sie sind robust gegenüber Rauschen und Ausreißern in den Daten.
- Skalierbarkeit: Sie funktionieren gut bei vielen Merkmalen und großen Datensätzen.

Beispiel:

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error

forest_model = RandomForestRegressor(random_state=1)
forest_model.fit(train_X, train_y)
melb_preds = forest_model.predict(val_X)
print(mean_absolute_error(val_y, melb_preds))
```

Das Ergebnis (z. B. MAE $\approx$ 191.670) ist oft deutlich besser als bei einem einzelnen Entscheidungsbaum (z. B. MAE $\approx$ 250.000).